

Dynamic

Video-127

Programming



Note :- This playlist is only for explanation of Dns & solutions.

See my "DP Concepts & Dns"
Playlist for understanding
DP from scratch...



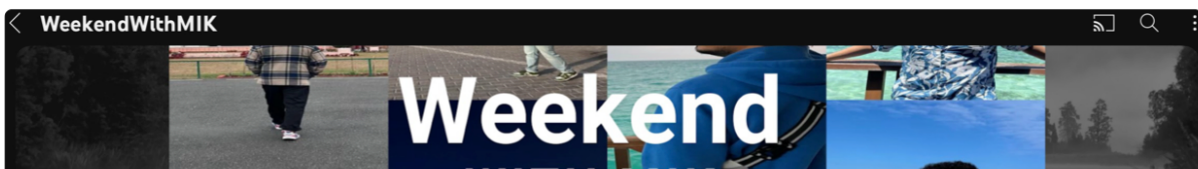
codestorywithmik



CSwithMIK



codestorywithMIK





Motivation :-

The best way to solve a big problem is to break it into smaller easy subproblems.

Rest everything will start making sense.



MIK...

3562. Maximum Profit from Trading Stocks with Discounts

Hard Topics Companies Hint

Profit

1	2	3	4
		105	

You are given an integer n , representing the number of employees in a company. Each employee is assigned a unique ID from 1 to n , and employee 1 is the CEO. You are given two **1-based** integer arrays, `present` and `future`, each of length n , where:

- `present[i]` represents the **current** price at which the i^{th} employee can buy a stock today.
- `future[i]` represents the **expected** price at which the i^{th} employee can sell the stock tomorrow.

The company's hierarchy is represented by a 2D integer array `hierarchy`, where `hierarchy[i] = [ui, vi]` means that employee u_i is the **direct boss** of employee v_i .

Additionally, you have an integer `budget` representing the **total funds** available for investment.

However, the company has a **discount policy**: if an employee's **direct boss** purchases their own stock, then the employee can buy their stock at **half** the original price ($\text{floor}(\text{present}[v] / 2)$).

Return the **maximum** profit that can be achieved without exceeding the given budget.

Note:

- You may buy each stock at most **once**.

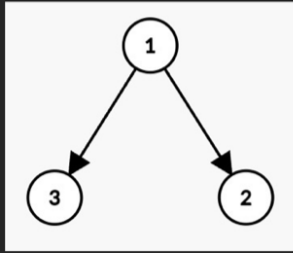


- You **cannot** use any profit earned from future stock prices to fund additional investments and must buy only from budget.

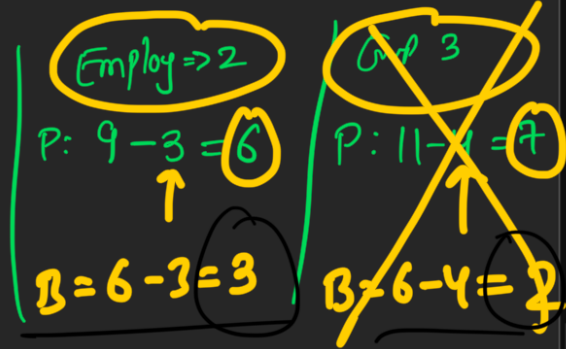
Input: $n = 3$, present = [4,6,8], future = [7,9,11], hierarchy = [[1,2],[1,3]], budget = 10

Output: 10

Explanation:



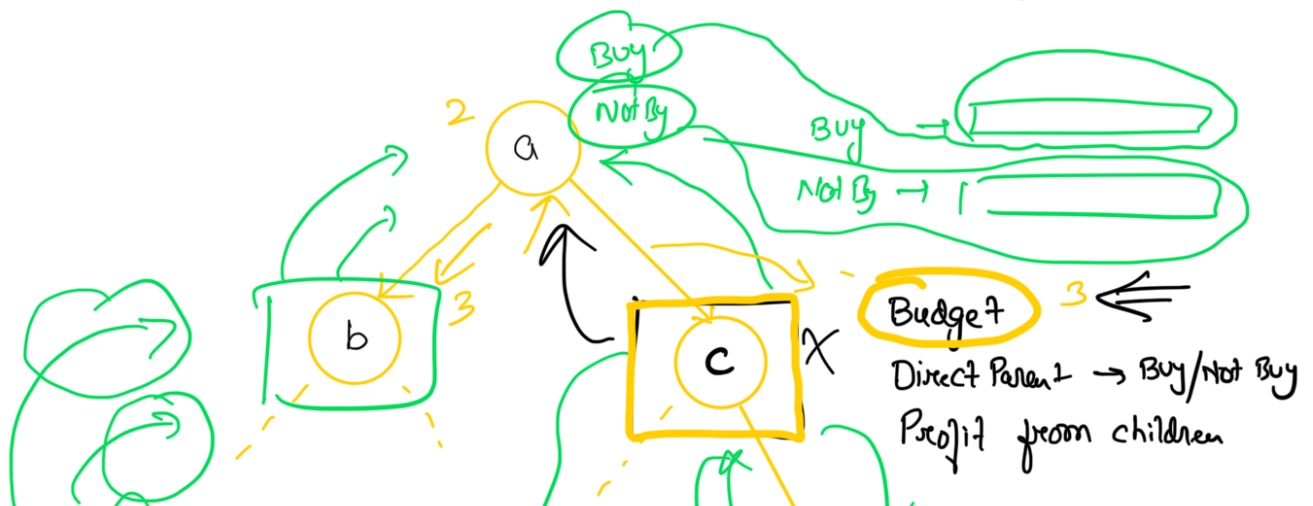
Budget = 10



- Employee 1 buys the stock at price 4 and earns a profit of $7 - 4 = 3$.
- Employee 3 would get a discounted price of $\text{floor}(8 / 2) = 4$ and earns a profit of $11 - 4 = 7$.
- Employee 1 and Employee 3 buy their stocks at a total cost of $4 + 4 = 8 \leq \text{budget}$. Thus, the maximum total profit achieved is $3 + 7 = 10$.

Thought Process

Boss(CEO) $\rightarrow$ Buy stock / not buy stock.



Budget = 8

$(U, \text{Buy/Not Buy}, \text{budget}) \rightarrow \text{max profit}$

$[U][1][b]$
 $[U][0][b]$
: 3D-vector

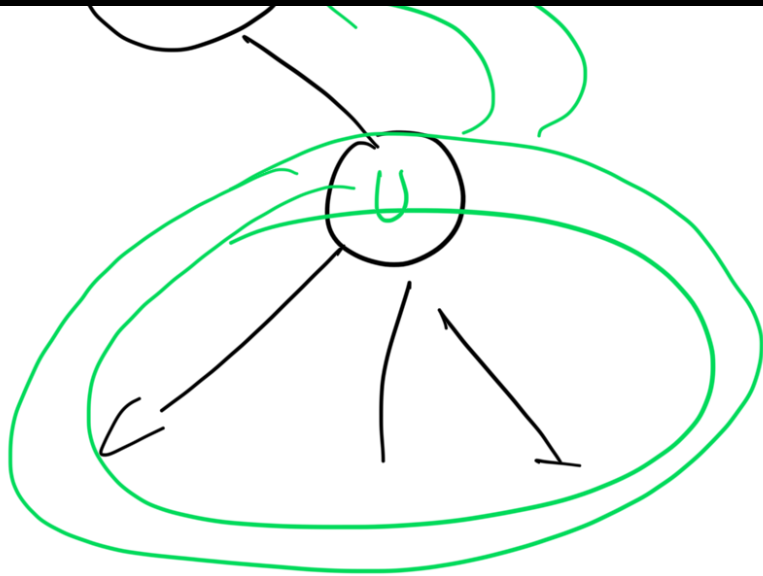
$[U][1] \rightarrow$

0	1	2	...	budget

$[U][0] \rightarrow$

0	1	2	...	budget

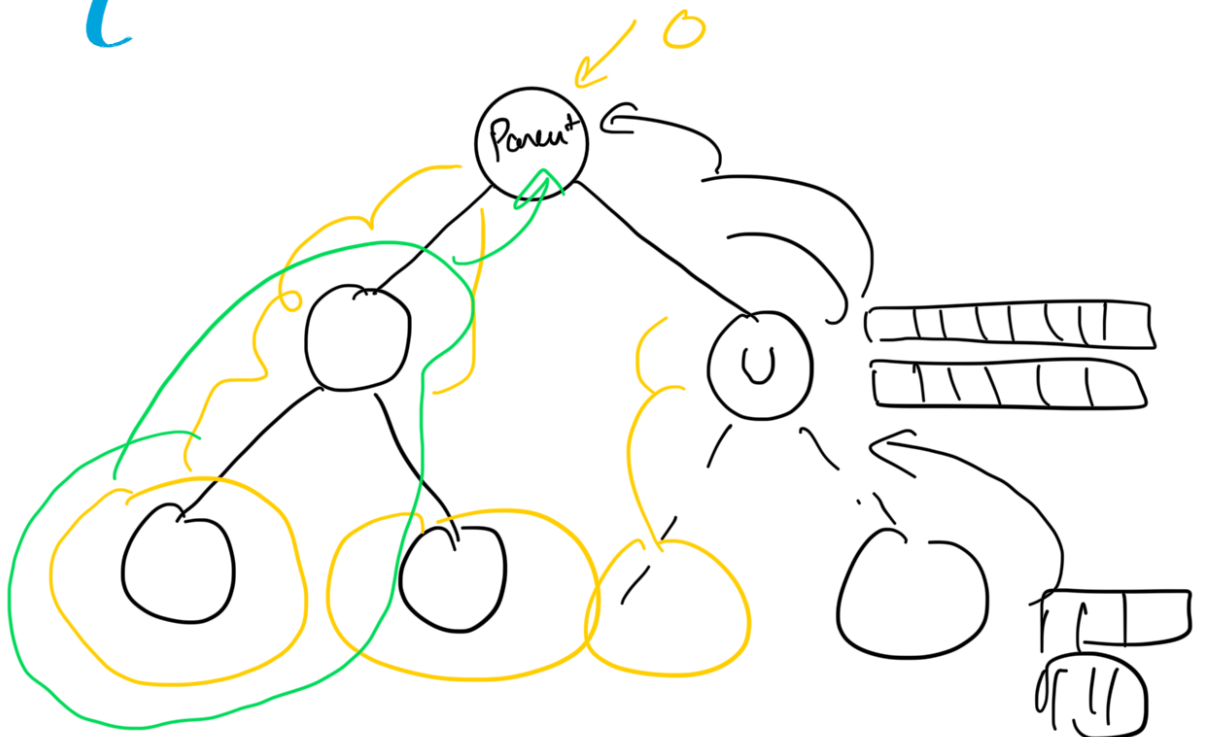
Paren² $\leftarrow$ Buy/Not



$$[u][0] = \begin{array}{|c|c|c|c|c|c|} \hline 0 & 1 & 2 & \dots & 6m. \\ \hline \end{array}$$

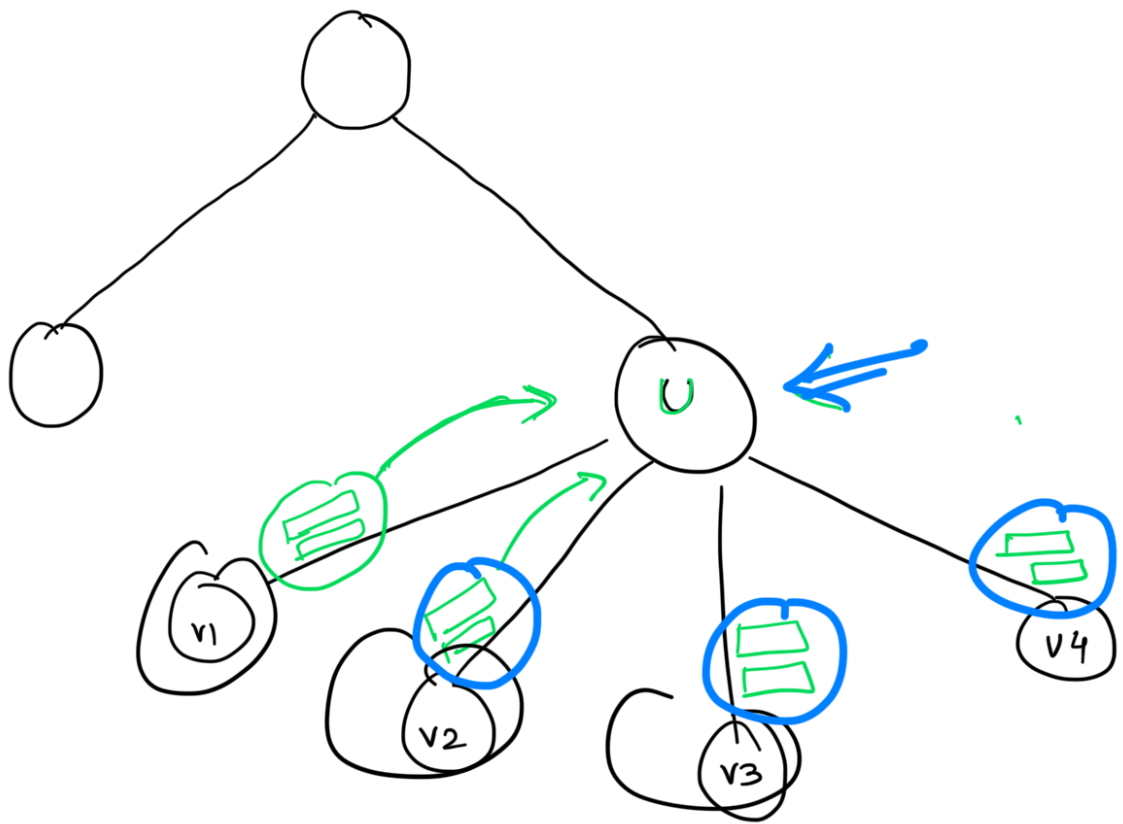
$[u][1] \text{ (budget)}$

how to proceed??



DFS

DFS(0, present, future, adj, StatesProfit, budget) {



$U \rightarrow \{ (\underline{\text{present}}, \underline{\text{future}}), (\underline{\text{present}}, \underline{\text{future}}) \}$

vector < pair < vector, vector > > ChildrenStatesProfit;

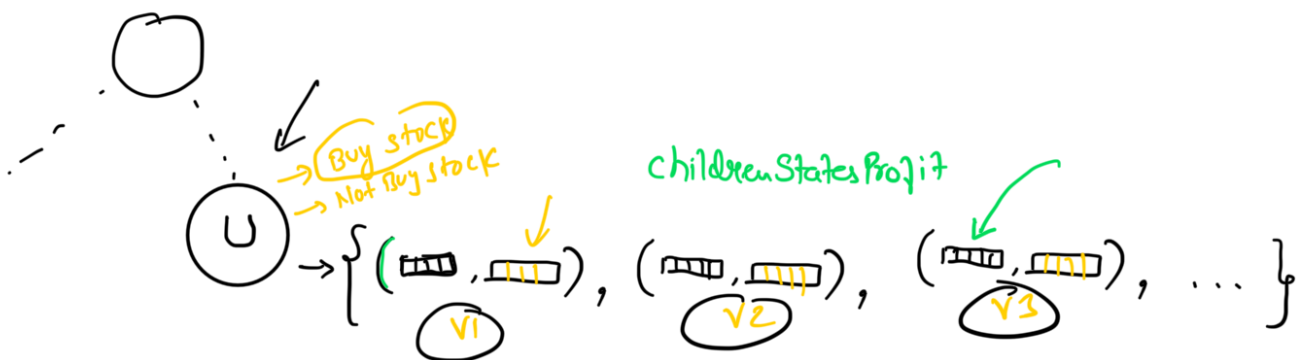
for (int &v : adj[u]) {

```

DFS (v, present, future, adj, statesProfit, budget);
childrenStatesProfit.push-back ( { statesProfit[v][0],
                                     statesProfit[v][1] } );
}

```

Calculating for U :-



```

Vector<int> bestProfitAtU (budget+1, 0);

```

```

Vector<int> childrenProfitIfUNotBought (budget+1, 0);

```

(, 0)

```
for (auto & child : childrenStatesProfit) {
    vector<int> temp (buy+1, 0);
```

budget used by $\leftarrow$ for (used = 0 ; used <= budget ; used++) {
prev children

budget for current $\leftarrow$ for (take = 0 ; take + used <= budget) {
children

temp[used + take] = max (temp[used + take] ,

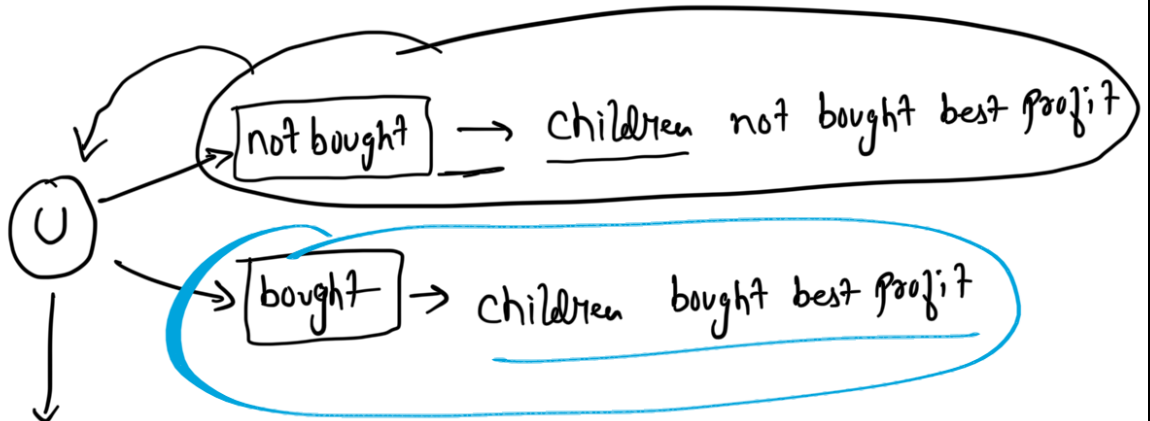
childrenProfitIfUNotBought [used] + child.first[take]) ;

}

}

childrenProfitIfUNotBought = temp;

}



bestProfitatU

for (int b = 0 ; b <= budget ; b++) {

bestProfitatU[b] = max (bestProfitatU[b] ,

Bought by U :-

Present[U] ;



for (int parent = 0 ; parent <= 1 ; parent++) {

Case-1 : U not bought stock

Case-2 : U buy stock

statesProfit[U][parent] = bestProfitAtU ;

}

Price = (Parent == 0) ? present[u] : parent[u]/2;

Profit = future[u] - Price;

//Case-2 When u bought stock

vector<int> childrenProfitIfUBought (budget+1, 0);

()

for (auto &child : childrenStatesProfit) {
vector<int> temp (buy+1, 0);

←

budget used by ← for (used = 0; used <= budget; used++) { ←
prev children

budget for current ← for (take = 0; take + used <= budget) { ←
children

temp[used+take] = max (temp[used+take],

childrenProfitIfUBought [used]

}

+ child->val[take]);

childrenProfitIfUBought = temp;

}

```
for (b = price; b <= budget; b++) {
```

```
    bestProfitatU[b] = max(bestProfitatU[b],
```

```
                           ChildrenProfitIfUBought[b-price]
```

```
                           + profit;
```

```
}
```

```
stateProfit[u][parent] = bestProfitatU;
```

```
}
```

1) FS.

